



TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN SISTEMAS DE INFORMACIÓN



Modernización del módulo Report Service en la plataforma BIC para el cliente ALSEA

Estudiante: Andrea Liñares Castro

Dirección: Fernando Silva Coira

Ángel Fernández Formoso

A Coruña, noviembre de 2025.

Dedicatoria

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Resumen

En el contexto de la gestión de procesos empresariales y la generación automática de informes ejecutivos, este Trabajo de Fin de Grado presenta la modernización del módulo Report Service para ALSEA, desarrollado por GBTEC Software AG. El proyecto aborda la optimización del flujo de generación de informes en formato Excel, mediante la incorporación de almacenamiento histórico, procesamiento asíncrono, control de duplicidad y mejoras en la experiencia de usuario. Para ello, se aplicó una metodología ágil iterativa e incremental, utilizando tecnologías como Spring Batch, Flyway, PostgreSQL, Elasticsearch y Angular. Las mejoras contribuyen a aumentar la escalabilidad, robustez y calidad del servicio, alineándose con las necesidades operativas y analíticas de ALSEA.

Abstract

Within the scope of business process management and automatic generation of executive reports, this undergraduate thesis presents the modernization of the Report Service module for ALSEA, developed by GBTEC Software AG. The project focuses on optimizing the report generation workflow in Excel format by incorporating historical storage, asynchronous processing, duplication control, and user experience improvements. An iterative and incremental agile methodology was applied, employing technologies such as Spring Batch, Flyway, PostgreSQL, Elasticsearch, and Angular. These enhancements contribute to increased scalability, robustness, and service quality, aligning with ALSEA's operational and analytical needs.

Palabras clave:

- Generación automática de informes
- Procesamiento asíncrono
- Metodología ágil
- Spring Batch
- PostgreSQL
- Elasticsearch
- Angular
- Control de duplicidad
- Migraciones de base de datos

Keywords:

- Automatic report generation
- Asynchronous processing
- Agile methodology
- Spring Batch
- PostgreSQL
- Elasticsearch
- Angular
- Duplication control
- Database migrations

Índice general

1	Introducción	1
1.1	Motivación	1
1.2	Objetivos	4
2	Fundamentos tecnológicos	5
2.1	Estado del arte	5
2.2	Tecnologías utilizadas	6
2.2.1	Web server	6
2.2.2	Web client	7
2.2.3	Database	7
2.2.4	Infraestructura y Despliegue	7
2.2.5	Herramientas de apoyo	8
3	Metodología y planificación	9
3.1	Metodología de desarrollo	9
3.1.1	Adaptación al proyecto	10
3.2	Planificación y seguimiento	10
4	Análisis	13
4.1	Requisitos	13
4.1.1	Actores	13
4.1.2	Requisitos funcionales	14
4.1.3	Requisitos no funcionales	15
4.1.4	Historias de usuario	15
4.2	Arquitectura del sistema	16
4.2.1	Cliente (Angular)	16
4.2.2	Servidor (Spring Boot y Spring Batch)	16
4.2.3	Capa de datos	16

4.3	Interfaz de usuario	17
5	Diseño	18
5.1	Arquitectura tecnológica del sistema	18
5.1.1	Visión general de la arquitectura	18
5.1.2	Comunicación entre componentes	18
5.1.3	Justificación de la arquitectura elegida	19
5.1.4	Stack tecnológico	19
5.2	Diseño de la aplicación	20
5.2.1	Cliente (Angular)	20
5.2.2	Servidor (Spring Boot + Spring Batch)	22
5.2.3	Capa de datos	24
5.3	Interacción cliente-servidor	26
5.3.1	Endpoints REST	27
5.3.2	Manejo de errores	28
6	Implementación y pruebas	30
6.1	Tareas de mejora	30
6.1.1	Vulnerabilidades	30
6.1.2	Coverage	31
6.1.3	Refactorización	31
6.1.4	Duplicación	31
6.2	Spring Batch discussion dificultades técnicas	31
6.3	Pruebas	31
6.3.1	Testing unitario	31
6.3.2	Testing de integración	31
6.3.3	Pruebas manuales y entorno de QA	31
7	Solución desarrollada	33
7.1	Introducción	33
7.2	Página de Generar	33
7.2.1	Funcionalidad	34
7.2.2	Diseño de la interfaz	34
7.2.3	Flujo de interacción	35
7.3	Página de Historial	36
7.3.1	Funcionalidad	36
7.3.2	Diseño de la interfaz	36
7.4	Release y publicación	37

8 Conclusión y trabajo futuro	39
8.1 Conclusiones	39
8.2 Posibles ampliaciones futuras	40
8.2.1 Actualización de la versión mayor de Angular	41
8.2.2 Mejoras en la gestión del historial	41
8.2.3 Refactorización adicional del código backend	43
8.2.4 Funcionalidades avanzadas	43
A Material adicional	45
Lista de acrónimos	47
Glosario	49
Bibliografía	51

Índice de figuras

1.1	Mapa de presencia internacional de ALSEA (Food Service Project).	1
1.2	Informe de tipo KPI con categoría de proceso Alternativa (todas las franquicias, sin rango de fechas).	3
1.3	Informe de tipo Producto con categoría de proceso Innovación (todas las franquicias, sin rango de fechas).	3
3.1	Distribución de horas trabajadas por mes durante el proyecto.	10
4.1	Interfaz original del Report Service antes de su modernización.	17
7.1	Interfaz principal del módulo de generación de informes.	35
7.2	Diagrama de flujo del proceso de generación de informes.	35
7.3	Vista de la página de historial de informes generados.	37
8.1	Mockup de la interfaz de historial con funcionalidad de eliminación de informes.	42
8.2	Diálogo de confirmación para la eliminación de informes.	42

Índice de cuadros

5.1	Atributos de la entidad <i>Report</i> en <i>alsea_reports.report</i>	26
-----	--	----

Capítulo 1

Introducción

1.1 Motivación

ALSEA, S.A.B. de C.V., es una empresa multinacional mexicana dedicada a la operación de marcas de restauración franquiciadas. Con presencia en América Latina y Europa, se ha consolidado como uno de los principales operadores del sector, gestionando cadenas tan conocidas como Starbucks, Burger King, Domino's Pizza, Vips o Foster's Hollywood. En España, su actividad se articula a través de Food Service Project, que engloba establecimientos de restauración rápida, cafeterías y restaurantes de comida casual.¹

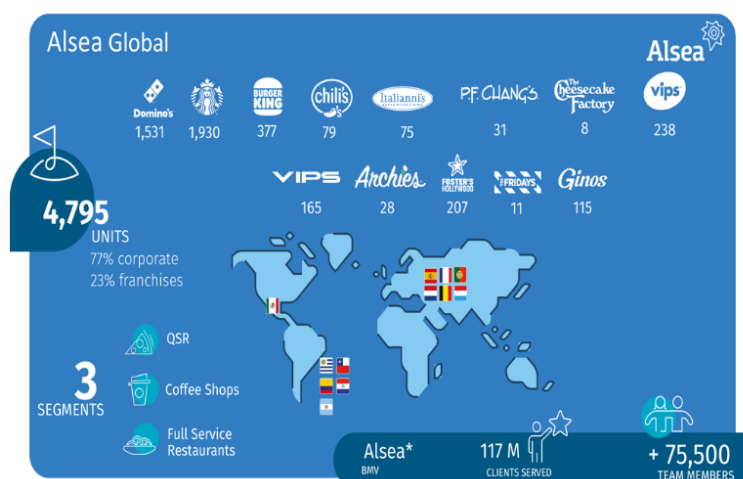


Figura 1.1: Mapa de presencia internacional de ALSEA (Food Service Project).

La magnitud de esta operación internacional hace que ALSEA maneje diariamente un volumen muy elevado de información relativa a ventas, operaciones, proyectos de innovación, indicadores de desempeño y actividades internas. En este contexto, disponer de datos fiables,

¹ Fuente: página web oficial de ALSEA, disponible en [Alsea Europa](#)

centralizados y fácilmente accesibles resulta fundamental para evaluar el rendimiento de las franquicias, identificar posibles desviaciones y tomar decisiones estratégicas fundamentadas. Sin herramientas de análisis adecuadas, la gestión de este conocimiento se vuelve compleja y poco eficiente.

Para dar respuesta a esta necesidad, GBTEC Software AG, compañía alemana especializada en soluciones de gestión de procesos de negocio (BPM), arquitectura empresarial (EAM) y gobierno, riesgo y cumplimiento (GRC), ofrece la *BIC Platform*, un ecosistema de software que permite modelar, ejecutar y analizar procesos empresariales. Dentro de este marco, se desarrolló para ALSEA un módulo específico denominado *Report Service*, cuya finalidad es transformar los datos de la plataforma de Process Execution en informes ejecutivos descargables en formato Excel.

La *BIC Platform* se despliega habitualmente en modalidad cloud (BIC Cloud) y está compuesta por varios módulos que cubren el ciclo de vida de los procesos: *Process Design* para modelar procesos mediante diagramas BPMN, y *Process Execution* para ejecutar y automatizar esos procesos, gestionando instancias, tareas y persistencia de variables. El *Report Service* extrae información de *Process Execution* (combinada con consultas a [Elasticsearch](#)) para componer informes Excel con KPI, [Retroplanning](#), [Heatmap](#) y otras métricas solicitadas por ALSEA.

El Report Service original ofrecía una solución sencilla y funcional: la interfaz permitía seleccionar el tipo de informe (KPI o Producto), la categoría (Innovación o Alternativa), una o varias franquicias y un rango de fechas. Con esta información, el backend recuperaba los datos correspondientes, generaba un libro de Excel con múltiples hojas y lo devolvía al usuario para su descarga local.

El sistema permite generar cuatro tipos de informes combinando el tipo (KPI o Producto) con la categoría del proceso (Innovación o Alternativa). Cada combinación produce un libro Excel con múltiples hojas que cubren distintos requisitos y métricas personalizadas solicitadas por el cliente. A continuación se muestran dos ejemplos representativos que ilustran la estructura y contenido de estos informes:

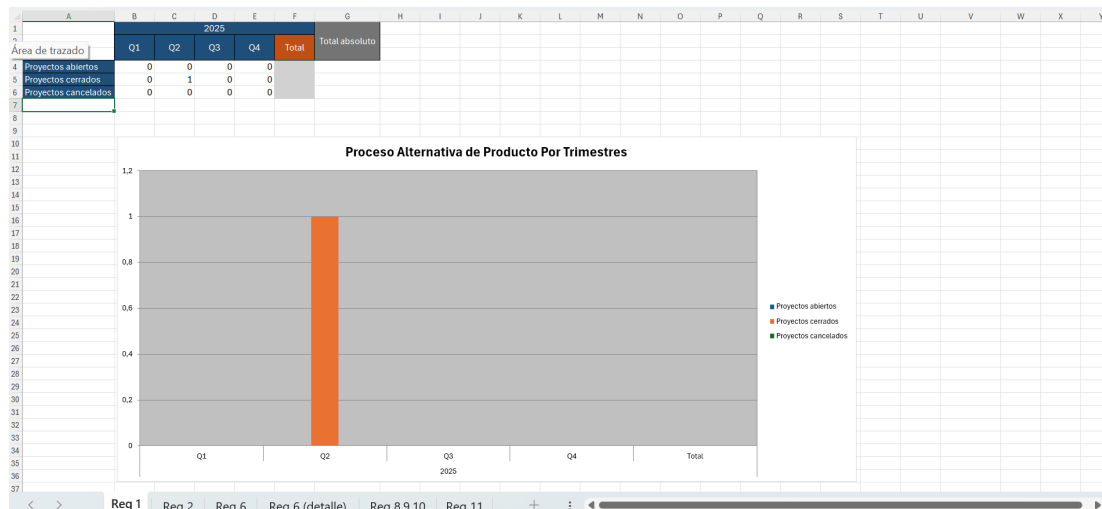


Figura 1.2: Informe de tipo KPI con categoría de proceso Alternativa (todas las franquicias, sin rango de fechas).

La captura muestra la hoja del Requisito 1, que presenta un **Histograma** con los procesos de alternativa de producto por trimestres, diferenciando proyectos abiertos, cerrados y cancelados, además de los totales y el total absoluto en 2025. En la parte inferior se observan las pestañas de otras hojas del libro, correspondientes a requisitos adicionales personalizados impuestos por el cliente.

A	B	C	D	E	F	G	H	I
Marca	Estado	Proyecto	Actividad	Responsable	Fecha de comienzo inicial	Fecha de comienzo actual	Fecha de finalización inicial	Fecha de finalización actual
Foster's Hollywood	ACTIVO	test_ana_6	Trabajosddp	DDP	04-jun-25		19-jun-25	
Foster's Hollywood	ACTIVO	test_ana_6	DOP2	DDP	19-jun-25		03-ago-25	
Starbucks España	ACTIVO	test_bruno	Estimacionventa	MKT	25-may-25		01-jun-25	
Starbucks España	ACTIVO	test_bruno	Comunicacionventa	LOGISTICA	01-jun-25		08-jun-25	
Starbucks España	ACTIVO	test_bruno	Caldedconsumo	GESTION DE PRODUCTO	08-jun-25		08-jun-25	
Starbucks España	ACTIVO	test_bruno	Estimacionventa	LOGISTICA	08-jun-25		22-jun-25	
Starbucks España	ACTIVO	test_bruno	Productosalmacen	DDP	22-jun-25		20-jul-25	
Starbucks Holand	ACTIVO	test_bruno_2	Mueques	GESTION DE PRODUCTO	19-abr-25		03-jun-25	
Starbucks Holand	ACTIVO	test_bruno_2	Catso	DDP	03-jun-25		10-jun-25	
Starbucks Holand	ACTIVO	test_bruno_2	DecisionMarca	MKT	10-jun-25		17-jun-25	
Starbucks Holand	ACTIVO	test_bruno_2	Estimacionventa	MKT	17-jun-25		24-jun-25	
Starbucks Portugal	ACTIVO	test_bruno_3	Focus	C_INTELIGENCE	24-jun-25		01-jul-25	
Starbucks Portugal	ACTIVO	test_bruno_3	Maguetas	GESTION DE PRODUCTO	14-may-25		04-jun-25	
Starbucks Portugal	ACTIVO	test_bruno_3	Catso	DDP	04-jun-25		11-jun-25	
Starbucks Portugal	ACTIVO	test_bruno_3	DecisionMarca	MKT	11-jun-25		18-jun-25	
Starbucks Portugal	ACTIVO	test_bruno_3	Internacional	MKT	18-jun-25		25-jun-25	
Starbucks Portugal	ACTIVO	test_bruno_3	Internacional	MKT	25-jun-25		02-jul-25	

Figura 1.3: Informe de tipo Producto con categoría de proceso Innovación (todas las franquicias, sin rango de fechas).

La captura muestra la hoja del heatmap actual, que representa para cada marca el estado de sus proyectos junto con las actividades, responsables que las llevan a cabo y fechas previstas. En la parte inferior se pueden observar las pestañas de otras hojas del libro, incluyendo

los retroplannings y una hoja de comparativa, que forman parte del conjunto de métricas y análisis solicitados por ALSEA.

Estos ejemplos muestran la riqueza informativa y el nivel de personalización de los informes generados por el Report Service, diseñados para cubrir las necesidades analíticas específicas de ALSEA en la gestión de sus procesos de innovación y alternativa de producto.

Sin embargo, con el paso del tiempo surgieron limitaciones notables. El sistema carecía de un almacenamiento histórico de informes, lo que obligaba a los usuarios a regenerarlos cada vez que deseaban acceder a información pasada. Además, se detectaron problemas de duplicación de código en el backend, ausencia de procesamiento asíncrono (lo que generaba bloqueos en la interfaz durante la creación de los informes) y una experiencia de usuario que no reflejaba plenamente las necesidades de la organización. Estas carencias constituyeron la motivación principal para emprender un proceso de mejora y modernización del Report Service.

1.2 Objetivos

El presente Trabajo de Fin de Grado documenta, analiza y evalúa las mejoras introducidas en el Report Service de ALSEA durante mis prácticas en GBTEC. En concreto, se plantearon los siguientes objetivos:

- Ampliar la funcionalidad del servicio con almacenamiento histórico en base de datos.
- Incrementar la mantenibilidad y calidad del código, reduciendo duplicaciones y aumentando la cobertura de pruebas.
- Optimizar la experiencia de usuario en la interfaz de generación de informes y acceso al historial.
- Introducir procesamiento asíncrono con [Spring Batch](#), evitando bloqueos en la interfaz.
- Registrar métricas de ejecución y tiempos de creación de informes.
- Añadir control de duplicidad para evitar regeneraciones innecesarias.
- Permitir la re-descarga de informes desde el historial.

Con ello, se pretende transformar el Report Service en una solución completa que cubra tanto la generación como la gestión del ciclo de vida completo de los informes ejecutivos.

Fundamentos tecnológicos

2.1 Estado del arte

Para la elaboración del estado del arte se han analizado distintas soluciones disponibles en el mercado relacionadas con la generación de informes empresariales y cuadros de mando, evaluando sus ventajas y limitaciones. El objetivo de este análisis es comprender cómo se abordan actualmente estas necesidades en el sector y en qué medida las herramientas existentes inspiran o contrastan con el desarrollo del módulo Report Service en ALSEA.

Microsoft Power BI¹ es una de las plataformas líderes en el ámbito de la inteligencia de negocio. Destaca por su capacidad de conexión a múltiples fuentes de datos y por ofrecer cuadros de mando interactivos y actualizados en tiempo real. Su principal fortaleza es la flexibilidad de visualización y la posibilidad de integrar gráficos avanzados sin conocimientos de programación. No obstante, entre sus limitaciones se encuentra la complejidad de configuración inicial y la necesidad de licencias específicas para acceder a todas las funcionalidades.

Tableau² constituye otra herramienta ampliamente utilizada. Su punto fuerte es la facilidad con la que permite generar visualizaciones intuitivas y estéticamente atractivas. Además, ofrece opciones de colaboración entre equipos y una buena integración con entornos corporativos. Sin embargo, su curva de aprendizaje puede ser elevada para usuarios sin experiencia previa, y su enfoque está más orientado a cuadros de mando interactivos que a informes exportables en formatos tradicionales como Excel.

Qlik Sense³ se centra en la exploración de datos y en la creación de dashboards personalizados. Su motor asociativo facilita el análisis de relaciones entre diferentes conjuntos de

¹<https://www.microsoft.com/es-es/power-platform/products/power-bi>

²<https://www.tableau.com/es-es/why-tableau/what-is-tableau>

³<https://www.qlik.com/es-es/products/qlik-sense>

datos, permitiendo descubrir patrones de forma ágil. Entre sus limitaciones destaca que, al igual que en el caso de Tableau, no siempre resulta la herramienta más adecuada para informes ejecutivos estandarizados en Excel, que siguen siendo demandados en muchos entornos corporativos.

Por otro lado, existen soluciones especializadas en automatización de informes Excel con plantillas predefinidas, aunque suelen carecer de personalización avanzada o integración con flujos de datos en tiempo real.

El módulo Report Service de ALSEA surge para cubrir este hueco: generar de forma sencilla y automatizada informes Excel ajustados a requisitos específicos de negocio (retroplanning, heatmaps, KPIs de innovación y alternativas), combinando la estandarización documental con la integración directa de datos de ejecución de procesos.

2.2 Tecnologías utilizadas

El desarrollo del Report Service se apoyó en un conjunto de tecnologías que abarcan tanto el cliente web como el servidor y la base de datos, junto con utilidades complementarias para despliegue, monitorización y calidad.

2.2.1 Web server

- **Spring Boot:** framework de desarrollo en Java que permite crear aplicaciones web y microservicios de forma rápida y estructurada. Aporta configuración automática, inyección de dependencias y un ecosistema rico de librerías.
- **Spring Batch:** módulo de Spring orientado al procesamiento por lotes. Se utilizó para implementar la generación asíncrona de informes, gestionando *jobs* y *steps* de manera controlada.
- **Hibernate / JPA:** framework de mapeo objeto-relacional que simplifica el acceso a la base de datos mediante entidades Java, evitando tener que escribir consultas SQL manuales.
- **Aspose Cells:** *Aspose Cells* es una librería de generación y manipulación de Excel utilizada para dar formato, estilos y estructura a los informes.
- **Elasticsearch:** motor de búsqueda utilizado como fuente de datos adicional, especialmente para recuperar información de instancias de procesos con consultas JSON flexibles.

- **Maven:** herramienta de gestión y automatización de proyectos Java, empleada para manejar dependencias, construir el proyecto y ejecutar tareas comunes.

2.2.2 Web client

- **Angular:** framework de desarrollo basado en TypeScript utilizado para construir la interfaz del usuario. Permite crear componentes modulares, mantener un enrutamiento dinámico y gestionar formularios y servicios de forma escalable.
- **Yarn:** gestor de dependencias para proyectos JavaScript que garantiza instalaciones reproducibles y seguras. Fue la herramienta principal para administrar librerías del frontend y realizar auditorías de seguridad.
- **Node.js:** entorno de ejecución necesario para compilar y ejecutar [Angular](#) en local, así como para el servidor de desarrollo.
- **TypeScript:** lenguaje de programación que extiende JavaScript con tipado estático y características avanzadas, facilitando el desarrollo de aplicaciones robustas y mantenibles.

2.2.3 Database

- **PostgreSQL:** sistema de gestión de bases de datos relacional, robusto y de código abierto, empleado para almacenar tanto la información de procesos como los informes generados.
- **Flyway:** herramienta de migraciones que asegura el versionado y la coherencia del esquema de base de datos en todos los entornos (desarrollo, pruebas y producción). Se utilizó para crear y mantener el esquema `alsea_reports` y su tabla principal `report`.
- **Testcontainers:** framework de testing que permite levantar contenedores ligeros de [PostgreSQL](#) en memoria durante la ejecución de pruebas, garantizando un entorno controlado y reproducible.

2.2.4 Infraestructura y Despliegue

- **Docker:** plataforma de contenedores utilizada para empaquetar tanto el backend como el frontend y garantizar despliegues reproducibles en diferentes entornos.
- **Git:** sistema de control de versiones empleado en combinación con ramas de funcionalidad y *Pull Requests*, asegurando trazabilidad y revisiones de código antes de su integración.

2.2.5 Herramientas de apoyo

- **Gestión de trabajo:** [Jira](#) se utilizó como tablero [Kanban](#) para organizar [EPICs](#), historias de usuario, subtarefas y gestionar prioridades, permitiendo un seguimiento visual del progreso de cada ticket.
- **Control de versiones y hosting:** [Git](#) para control de versiones y [GitHub](#) como repositorio remoto centralizado donde se alojaba el código fuente del proyecto, facilitando la colaboración mediante Pull Requests y revisiones de código.
- **Flujo de trabajo Git:** [Gitflow](#) [1] se adoptó como convención de ramas (feature/release/hotfix) para mantener estabilidad en las ramas de producción y organizar el desarrollo de forma estructurada.
- **Integración continua / CI & CD:** [GitHub Actions](#) se configuró con acciones personalizadas para automatizar el build del proyecto, ejecutar pruebas unitarias e integradas, y realizar despliegues. Los pipelines se integraron con [SonarQube](#) ([SonarCloud](#)) para validar métricas de calidad de código (cobertura, duplicidad, code smells) antes de aceptar cada [PR](#).
- **Registro y gestión de artefactos:** [Quay](#) se empleó como registry principal para almacenar y versionar las imágenes Docker de los entornos Ansible de prueba, mientras que [JFrog Artifactory](#) sirvió como repositorio de artefactos binarios y dependencias Maven/NPM, facilitando la distribución controlada de versiones.
- **Orquestación de despliegue y automatización:** [Ansible](#) (gestionado mediante [AWX](#)) permitió ejecutar playbooks para levantar entornos de prueba y automatizar despliegues en [QA](#) e integración.
- **Entornos de desarrollo:** [IntelliJ IDEA](#) para backend (Java/Spring) y [Visual Studio Code](#) para frontend (Angular/TypeScript).
- **Pruebas y depuración:** [Postman](#) para pruebas de API y [Google Chrome](#) para pruebas manuales en entorno local.
- **Calidad de código:** [SonarCloud](#) / [SonarQube](#) se integró en los pipelines de CI para realizar análisis estático de código, medir cobertura de pruebas, detectar duplicaciones y controlar la deuda técnica, garantizando que cada cambio cumpliera con los umbrales de calidad definidos por la empresa.

Metodología y planificación

3.1 Metodología de desarrollo

El proyecto se llevó a cabo siguiendo una metodología ágil [2] ligera basada en iteraciones cortas y gestión Kanban [3] en [Jira](#). Este enfoque resultó adecuado por la naturaleza incremental de las mejoras del Report Service, la necesidad de integrar cambios en backend y frontend con validación continua, y la conveniencia de priorizar asuntos operativos (bugs y chores) junto con funcionalidades nuevas. Jira se configuró con un tablero Kanban que permitía visualizar el flujo de trabajo completo, con estados definidos (To Do → Task Breakdown → In Progress → Ready for QA/PR → QA/PR OK → Done) para cada tarea, facilitando el seguimiento de prioridades y la identificación de bloqueos.

Las características principales del enfoque fueron:

- **Iterativo e incremental** [4]: cada conjunto de cambios (feature/refactor/fix/chore) se abordó como una tarjeta en el tablero, produciendo un incremento funcional verificable (código, tests y PR).
- **Retroalimentación continua**: dailies diarias con el equipo para sincronizar prioridades, desbloquear tareas y validar entregables; revisión de PRs con comentarios técnicos.
- **Flexibilidad**: replanificación rápida en función de prioridades del [EPIC](#) y de incidencias detectadas (p. ej., compatibilidad de datos, fechas nulas, duplicidades de informe).
- **Calidad integrada**: pruebas unitarias e integradas, migraciones controladas con [Flyway](#), y verificación manual desde Postman/Chrome antes de integrar los cambios en el frontend.

3.1.1 Adaptación al proyecto

La incorporación al proyecto siguió una fase breve de formación, alineada con las tecnologías clave:

- **Formación inicial:** cursos introductorios de Spring Batch, [Docker](#), Patrones de Diseño en Java y Angular (priorizando primero backend) proporcionados por la suscripción mediante la empresa de la entidad emisora [Udemy](#).
- **Aterrizaje en Jira:** asignación al [EPIC](#) "Improvements for ALSEA Project in 2025 Q3 & Q4", con tareas tipificadas como Story, Task o Bug, cada una con subtareas y prioridad.
- **Flujo de trabajo:**
 - Rama por tarea (feature/fix branch) en Git.
 - Commits atómicos y descriptivos.
 - Pull Request hacia `development`, con code review y checks automáticos de SonarCloud y build de pull/push.
 - Cierre de la tarea al completar [PR](#) + validación.

El orden de adaptación técnica fue: backend → datos → frontend, permitiendo una integración progresiva.

3.2 Planificación y seguimiento

El trabajo se desarrolló durante tres meses de prácticas:

- Julio: 7 h/día, 5 días/semana.
- Agosto: 7 h/día, 5 días/semana.
- Septiembre: 6 h/día, 5 días/semana.

La dedicación horaria total del proyecto se muestra en la figura siguiente:

Año	Mes	Tiempo total (h)	Tiempo teórico (h)
2025	Septiembre	121h 0min	126h 0min
2025	Agosto	140h 15min	140h 0min
2025	Julio	148h 43min	147h 0min

Figura 3.1: Distribución de horas trabajadas por mes durante el proyecto.

Como se observa en la figura 3.1, la distribución del esfuerzo fue relativamente equilibrada a lo largo de los tres meses. Julio y agosto concentraron 7 horas diarias, mientras que en septiembre la dedicación se redujo a 6 horas diarias debido a las normativas de la organización de las prácticas, al tener que compaginar con el inicio del curso universitario.

La planificación se estructuró en etapas en lugar de Sprints formales, manteniendo la dinámica ágil con dailies y tablero Kanban:

- **Onboarding y formación dirigida (principios de julio)**

- Cursos: Spring Batch, Docker.
- Puesta a punto de entorno local y perfiles.

- **Limpieza y refactors (chores) (julio)**

- Reducción de duplicidad de código (utils de Excel, fechas, estilos).
- Reorganización de paquetes y utilidades comunes.
- Mejora de tests unitarios en servicios clave y aumento del coverage del proyecto para llegar a los checks mínimos de la empresa en SonarCloud.

De esta manera, empezando con tareas de menor complejidad, se fue ganando confianza en el código base y las herramientas, preparando el terreno para las funcionalidades principales.

- **Evolución backend (núcleo) (final de julio y agosto)**

- Flyway: adición de esquema a `sea_reports` y tabla `report`.
- Persistencia de informes y endpoints de consulta/descarga.
- Spring Batch: implementación de la generación asíncrona de informes.
- Probe de duplicidad: comprobación del bloqueo de informes idénticos en menos de 5 min.
- Métricas de tiempos: registro para soporte y monitorización.
- Pruebas durante todo el desarrollo.

- **Formación Angular + integración frontend (principios de septiembre)**

- Curso breve de Angular.
- Implementación de la página de Historial con re-descarga de informes.
- Ajustes en la página del generador con las validaciones pertinentes en cada comprobación.

- Paginación/navegación y metadatos (categoría, tipo, franquicias, fechas).
- **Cierre, release y documentación (última semana de septiembre)**
 - Preparación de release y publicación de la nueva versión.
 - Actualización de README (frontend/back) y guía de uso.
 - Recopilación de evidencias (PRs, capturas, pruebas).

Capítulo 4

Análisis

4.1 Requisitos

4.1.1 Actores

En el sistema intervienen distintos actores y componentes que participan en la generación, almacenamiento y consulta de los informes ejecutivos:

- **Usuario ALSEA:** empleado o analista encargado de generar y consultar los informes a través de la interfaz web. Puede lanzar nuevas generaciones, visualizar el historial y volver a descargar informes anteriores.
- **Aplicación web (Web Client):** cliente desarrollado en Angular que permite la interacción del usuario con el sistema. Presenta formularios, mensajes de estado y botones de acción.
- **Servicio de reportes (Web Server):** aplicación de servidor desarrollada con Spring Boot. Gestiona la lógica de negocio, coordina la generación asíncrona de informes, controla la duplicidad y persiste los resultados.
- **Origen de datos de procesos:** base de datos de ejecución de procesos de BIC (BIC Process Execution) y fuentes adicionales en *Elasticsearch*, de donde se obtienen las variables necesarias para la composición de los informes.
- **Base de datos de informes:** sistema PostgreSQL donde se almacenan los informes generados, junto con sus metadatos (categoría, tipo, fechas, franquicias, tamaño, tiempo de ejecución, etc.).

4.1.2 Requisitos funcionales

Los requisitos funcionales definen el comportamiento observable del sistema y las acciones que el usuario puede realizar:

1. Generación de informes:

- El sistema debe permitir la generación de informes en formato Excel, ofreciendo la posibilidad de seleccionar los siguientes parámetros: *tipo* (KPI o Producto), *categoría* (Innovación o Alternativa), *franquicia(s)* —una, varias o todas— y *rango de fechas* (opcional), que por defecto abarcará desde el inicio de la disponibilidad de datos.
- La generación se debe realizar de forma **asíncrona**, sin bloquear la interfaz.
- La aplicación debe mostrar mensajes informativos del estado de la generación: “*Comprobando solicitud*”, “*Generando informe*”, “*Descarga iniciada*”, “*Error de red*” o “*Informe reciente disponible en la pestaña Historial*”.

2. Control de duplicidad:

- Antes de generar un nuevo informe, el sistema debe comprobar si existe un informe idéntico generado recientemente (p. ej., en los últimos 5 minutos).
- En caso afirmativo, se debe bloquear la nueva generación y notificar al usuario que el informe ya está disponible en el historial.
- Para realizar esta comprobación, se genera una **firma única** basada en los parámetros de entrada.

3. Descarga y re-descarga de informes:

- Al finalizar una generación válida, el informe debe descargarse automáticamente en el navegador del usuario.
- El informe debe almacenarse en la base de datos para su consulta futura.
- El usuario podrá volver a descargar cualquier informe desde la pestaña *Historial*.

4. Historial de informes:

- El sistema debe disponer de una página de *Historial* que muestre todos los informes generados, junto con sus metadatos: fecha de creación, categoría de proceso, tipo de informe, franquicias a las que hace referencia (marcas) y rango de fechas.
- El historial debe estar ordenado de forma descendente (más recientes primero).
- Debe implementarse paginación mediante botones “*5 anteriores*” y “*5 siguientes*”.

4.1.3 Requisitos no funcionales

Los requisitos no funcionales garantizan la calidad y las propiedades del sistema:

- **Usabilidad:** interfaz simple e intuitiva, con botones y mensajes claros que guíen al usuario en todo momento.
- **Rendimiento:** la ejecución de informes no debe bloquear la aplicación; la respuesta del historial debe ser inmediata.
- **Confiabilidad:** almacenamiento persistente en base de datos; control de duplicidad para evitar sobrecarga.
- **Mantenibilidad:** estructura modular en capas (controladores, servicios, utilidades) y cobertura de pruebas unitarias e integradas.
- **Escalabilidad:** posibilidad de ejecutar múltiples trabajos de generación en paralelo mediante Spring Batch y Docker.
- **Seguridad:** control de acceso a endpoints y protección del contenido de los informes.
- **Observabilidad:** registro de métricas, logs estructurados y seguimiento de procesos.
- **Compatibilidad:** funcionamiento correcto en navegadores modernos y exportación a formatos Excel estándar.

4.1.4 Historias de usuario

A continuación se detallan las principales historias de usuario definidas para el proyecto:

- **HU01 – Generar informe:** Como usuario, quiero seleccionar los parámetros de tipo, categoría, franquicias y rango de fechas para generar un informe Excel personalizado.
- **HU02 – Visualizar estados de generación:** Como usuario, quiero recibir notificaciones sobre el estado del informe (comprobando, generando, completado, error, duplicado) para conocer el progreso.
- **HU03 – Control de duplicidad:** Como usuario, quiero que el sistema detecte y bloquee la generación de informes idénticos para evitar duplicidades innecesarias.
- **HU04 – Descarga automática:** Como usuario, quiero que el informe se descargue automáticamente una vez generado.
- **HU05 – Historial de informes:** Como usuario, quiero poder consultar un listado con todos los informes generados, ver su información.

- **HU06 – Re-descarga:** Como usuario, quiero volver a descargar un informe anterior sin tener que generarlo de nuevo.

4.2 Arquitectura del sistema

La aplicación sigue una arquitectura **cliente-servidor** basada en servicios web **REST** y procesamiento asíncrono.

4.2.1 Cliente (Angular)

El cliente web, desarrollado en Angular, gestiona la interacción del usuario y las peticiones al servidor:

- Página principal: módulo *Report Generator*, donde se configuran los parámetros y se lanza la generación.
- Página secundaria: módulo *Report History*, donde se visualizan y descargan los informes existentes.
- Comunicación con el servidor mediante servicios **HTTP**, con notificaciones de estado y validaciones de entrada.

4.2.2 Servidor (Spring Boot y Spring Batch)

El servidor gestiona la lógica principal:

- Controladores REST que reciben solicitudes de generación.
- Servicio de duplicidad (*Probe*) que calcula una firma única de parámetros.
- Ejecución asíncrona mediante **Spring Batch**, con jobs y steps configurables.
- Persistencia de resultados y metadatos en **PostgreSQL**.
- Generación de ficheros Excel con **Aspose Cells**.

4.2.3 Capa de datos

- Fuentes de información: **BIC Process Execution** y **Elasticsearch**, de las que se extraen las variables necesarias.
- Almacenamiento: **PostgreSQL** con migraciones controladas por **Flyway**.

4.3 Interfaz de usuario

La interfaz de usuario original del Report Service presentaba un diseño sencillo y funcional, pero con limitaciones evidentes desde el punto de vista de la experiencia de usuario. Se trataba de una única página web donde el usuario podía configurar los parámetros básicos del informe: tipo (KPI o Producto), categoría (Innovación o Alternativa), selección de franquicias y un rango de fechas opcional.

Una vez configurados estos parámetros, el usuario podía solicitar la generación del informe mediante un botón de descarga. El sistema procesaba la solicitud de forma síncrona, bloqueando la interfaz durante todo el proceso de generación, lo que podía resultar en tiempos de espera prolongados sin retroalimentación clara sobre el estado del proceso. Al finalizar, el informe Excel se descargaba directamente al navegador del usuario.

El diseño inicial, aunque funcional, presentaba carencias de usabilidad y no incluía historial ni procesamiento asíncrono. Las principales limitaciones eran: ausencia de historial, falta de retroalimentación durante la generación, procesamiento síncrono que bloqueaba la interfaz, diseño anticuado y ausencia de control de duplicidad. Estas carencias motivaron el desarrollo de una nueva interfaz modernizada (capítulo 7).

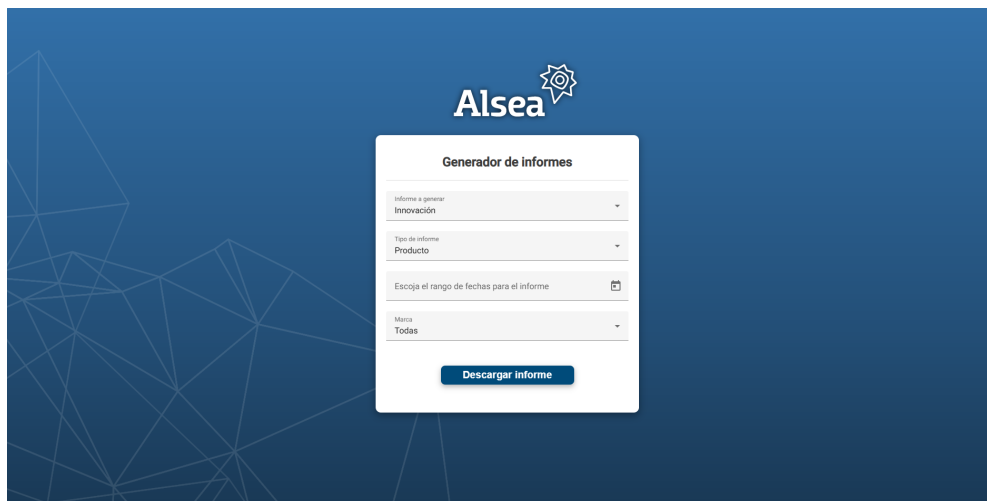


Figura 4.1: Interfaz original del Report Service antes de su modernización.

Capítulo 5

Diseño

5.1 Arquitectura tecnológica del sistema

En este apartado se describe la estructura final elegida para el Report Service modernizado, justificando las decisiones técnicas tomadas y explicando cómo interactúan los distintos componentes del sistema.

5.1.1 Visión general de la arquitectura

El sistema sigue una arquitectura **cliente-servidor de tres capas** que separa claramente las responsabilidades y facilita el mantenimiento y escalabilidad:

- **Capa de presentación (Cliente):** aplicación web desarrollada en Angular 15 que proporciona la interfaz de usuario para configurar, generar y consultar informes.
- **Capa de lógica de negocio (Servidor):** backend basado en Spring Boot 3 y Spring Batch que gestiona las peticiones, coordina la generación asíncrona de informes, valida duplicidades y persiste los resultados.
- **Capa de datos:** base de datos PostgreSQL que almacena los informes generados y sus metadatos, complementada con Elasticsearch como fuente de datos para consultar variables de procesos.

5.1.2 Comunicación entre componentes

La comunicación entre cliente y servidor se realiza mediante una [API REST](#) sobre [HTTP](#), intercambiando datos en formato [JSON](#). Los principales endpoints implementados son:

- `POST /api/reports/generate`: inicia la generación de un nuevo informe. Recibe como parámetros el tipo, categoría, lista de franquicias y rango de fechas opcional.

- GET `/api/reports/history`: devuelve el listado paginado de informes generados, ordenados de más reciente a más antiguo.
- GET `/api/reports/{id}/download`: descarga un informe específico por su identificador `UUID`.
- POST `/api/reports/probe`: verifica si existe un informe idéntico generado recientemente (control de duplicidad).

El servidor responde con códigos `HTTP` estándar (200 OK, 201 Created, 400 Bad Request, 404 Not Found, 409 Conflict para duplicados) y mensajes `JSON` estructurados que incluyen estado, datos y mensajes de error cuando corresponde.

5.1.3 Justificación de la arquitectura elegida

La arquitectura seleccionada ofrece múltiples ventajas:

- **Modularidad**: cada capa tiene responsabilidades bien definidas. El cliente gestiona la presentación, el servidor la lógica de negocio y la capa de datos la persistencia.
- **Separación de responsabilidades**: el uso de Spring Batch permite aislar la lógica de generación asíncrona de informes del flujo principal de peticiones `REST`, evitando bloqueos y mejorando la experiencia de usuario.
- **Escalabilidad**: la arquitectura permite escalar horizontalmente el backend (múltiples instancias detrás de un balanceador) y verticalmente la base de datos según las necesidades.
- **Mantenibilidad**: el patrón de capas y la inyección de dependencias de Spring facilitan pruebas unitarias, refactorizaciones y evolución del código.
- **Trazabilidad**: el uso de Flyway para migraciones de base de datos garantiza que los cambios en el esquema estén versionados y controlados.
- **Observabilidad**: Spring Actuator proporciona endpoints de monitorización (`/actuator/health`, `/actuator/metrics`) que permiten supervisar el estado del sistema.

5.1.4 Stack tecnológico

El stack tecnológico final adoptado es:

- **Cliente**: Angular 15 + TypeScript + Yarn (gestión de dependencias).
- **Servidor**: Spring Boot 3 + Spring Batch + Hibernate/JPA + Maven.

- **Almacenamiento:** PostgreSQL (informes) + Elasticsearch (variables de procesos).
- **Infraestructura:** Docker (contenedores) + Ansible (despliegue) + [AWX](#) (orquestración).
- **Migraciones:** Flyway para control de versiones del esquema de base de datos.
- **Generación de Excel:** Aspose Cells para construcción y formato de workbooks.
- **Logs y monitorización:** Spring Actuator + Elasticsearch (almacenamiento [On-Premise](#) de logs).

5.2 Diseño de la aplicación

A continuación se detalla el diseño interno de los principales componentes del sistema, describiendo su estructura, patrones aplicados y flujos de datos.

5.2.1 Cliente (Angular)

El cliente web está estructurado siguiendo las mejores prácticas de Angular, organizando el código en módulos, componentes y servicios reutilizables.

Estructura de módulos y componentes

La aplicación se divide en los siguientes módulos principales:

- **ReportGeneratorModule:** módulo que contiene el componente principal de generación de informes (`ReportGeneratorComponent`). Incluye formularios reactivos para capturar los parámetros (tipo, categoría, franquicias, fechas) y lógica para invocar el servicio de generación.
- **ReportHistoryModule:** módulo que agrupa el componente de historial (`ReportHistoryComponent`) y su tabla paginada. Gestiona la visualización de informes previos y permite la re-descarga.
- **SharedModule:** módulo compartido que exporta componentes y directivas reutilizables (spinners de carga, mensajes de notificación, validadores personalizados, pipes de formato).

Servicios Angular

Los servicios encapsulan la lógica de comunicación con el backend y la gestión de estado:

- **report.service.ts**: servicio principal que implementa los métodos para interactuar con la [API REST](#):
 - `generateReport(params)`: envía petición POST a `/api/reports/generate`.
 - `getHistory(page, size)`: obtiene el listado paginado de informes.
 - `downloadReport(id)`: descarga un informe por su ID.
 - `checkDuplicate(params)`: verifica duplicidad mediante `/api/reports/probe`.
- **notification.service.ts**: gestiona la visualización de mensajes informativos, de error o de éxito al usuario mediante toasts o snackbars.
- **loading.service.ts**: controla el estado de carga global de la aplicación, mostrando u ocultando spinners durante las peticiones asíncronas.

Flujo de datos entre componentes

El flujo típico de generación de un informe es:

1. El usuario rellena el formulario en `ReportGeneratorComponent`.
2. Al pulsar "Generar", el componente valida los datos y llama a `report.service.generateReport`.
3. El servicio primero invoca `checkDuplicate()` para verificar si existe un informe reciente idéntico.
4. Si no hay duplicado, se lanza la generación. El servicio muestra un spinner mediante `loading.service`.
5. Cuando el backend responde, se descarga automáticamente el fichero y se muestra un mensaje de éxito con `notification.service`.
6. Si hay duplicado, se notifica al usuario para que consulte el historial.

Validaciones y feedback al usuario

El formulario de generación implementa validaciones reactivas:

- Campos obligatorios: tipo, categoría y al menos una franquicia.
- Validación de fechas: si se especifica rango, la fecha de fin debe ser posterior a la de inicio.
- Mensajes contextuales: cada campo muestra ayudas inline si hay errores.

Durante la generación, se muestran mensajes de estado: "Comprobando solicitud", "Generando informe", "Descarga iniciada" o "Informe reciente disponible en Historial".

Frameworks y librerías de interfaz

El cliente utiliza:

- **Angular Material**: biblioteca de componentes UI que proporciona formularios, tablas, botones, diálogos y snackbars con diseño Material Design.
- **RxJS**: para gestión reactiva de flujos asíncronos (Observables) en servicios y componentes.
- **Yarn**: gestor de dependencias para instalación reproducible de paquetes.

5.2.2 Servidor (Spring Boot + Spring Batch)

El backend está organizado en capas siguiendo el patrón arquitectónico de **capas (Layered Architecture)** con inyección de dependencias gestionada por Spring.

Estructura de paquetes

La aplicación se organiza en los siguientes paquetes Java:

- **com.gbtec.bic.alsea.reports.controller**: controladores [REST](#) que exponen los endpoints de la [API](#).
- **com.gbtec.bic.alsea.reports.service**: servicios que implementan la lógica de negocio (generación, consulta, duplicidad).
- **com.gbtec.bic.alsea.reports.repository**: repositorios JPA que gestionan el acceso a la base de datos.
- **com.gbtec.bic.alsea.reports.batch**: configuración de jobs y steps de Spring Batch.
- **com.gbtec.bic.alsea.reports.model**: entidades JPA y DTOs.
- **com.gbtec.bic.alsea.reports.config**: clases de configuración (beans, propiedades, seguridad).
- **com.gbtec.bic.alsea.reports.util**: utilidades comunes (generadores Excel, cálculo de firmas, formateo de fechas).

Patrón de diseño y arquitectura

El backend aplica los siguientes patrones:

- **Arquitectura en capas:** separación clara entre controladores (presentación), servicios (lógica de negocio) y repositorios (acceso a datos).
- **Inyección de dependencias:** Spring gestiona el ciclo de vida de los beans y las dependencias mediante anotaciones (@Autowired, @Service, @Repository).
- **DTOs (Data Transfer Objects):** los controladores reciben y devuelven DTOs en lugar de entidades JPA, evitando exponer detalles internos del modelo de dominio.
- **Builder pattern:** para construcción fluida de objetos complejos (parámetros de jobs, configuración de Excel).

Flujo de generación de informes

El flujo completo de generación de un informe es:

1. **ReportController** recibe la petición POST en `/api/reports/generate` con los parámetros en formato **JSON**.
2. El controlador valida los datos de entrada y delega en **ReportService**.
3. **ReportService** invoca a **ProbeService** para comprobar duplicidad:
 - Se calcula una firma única (hash MD5) a partir de tipo, categoría, franquicias y fechas.
 - Se busca en la base de datos si existe un informe con esa firma generado en los últimos 5 minutos.
 - Si existe, se devuelve un código 409 Conflict con el ID del informe duplicado.
4. Si no hay duplicado, **ReportService** lanza un **Spring Batch Job** de forma asíncrona mediante **JobLauncher**.
5. El **Job** de Spring Batch consta de un único **Step** que:
 - Lee variables de procesos desde Elasticsearch y desde la base de datos de BIC Process Execution.
 - Procesa los datos aplicando cálculos, agregaciones y transformaciones según el tipo de informe.

- Genera el workbook Excel mediante **ExcelGenerator** (que utiliza Aspose Cells para aplicar formatos, estilos, heatmaps, retroplannings, etc.).
 - Persiste el informe en PostgreSQL mediante **ReportRepository**, almacenando metadatos y contenido binario.
6. Al finalizar el Job, se registran métricas de tiempo de ejecución y tamaño del fichero.
 7. El controlador devuelve el informe generado al cliente como stream binario con cabeceras `Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet` y `Content-Disposition: attachment`.

Diseño del Job y Step de Spring Batch

El job de generación (`GenerateReportJob`) está configurado con:

- **Parámetros de entrada:** tipo de informe, categoría, franquicias, fechas, ID del informe a generar.
- **Step único** (`GenerateReportStep`):
 - **Reader:** no utiliza reader estándar; en su lugar, el `Tasklet` lee directamente de Elasticsearch y PostgreSQL.
 - **Processor:** lógica de transformación y cálculo de métricas (**KPIs**, distribuciones trimestrales, estados de proyectos).
 - **Writer:** generación del Excel mediante Aspose y persistencia en base de datos.
- **Listeners:** registran inicio, fin y errores del job para monitorización y logs.

El uso de Spring Batch permite:

- Ejecución asíncrona sin bloquear el thread principal.
- Reintentos automáticos en caso de fallos transitorios.
- Trazabilidad completa de cada ejecución (timestamps, estado, parámetros).

5.2.3 Capa de datos

El modelo de datos está diseñado para soportar el almacenamiento eficiente de informes y facilitar consultas rápidas por metadatos.

Modelo entidad-relación

El esquema `alsea_reports` contiene una única tabla principal `report`, gestionada mediante migraciones de Flyway. La entidad JPA `ReportEntity` mapea esta tabla con los siguientes atributos:

- `id` (UUID): clave primaria generada automáticamente.
- `file_name` (String): nombre del fichero Excel.
- `report_type` (enum): tipo de informe (KPI / PRODUCT).
- `report_category` (enum): categoría de proceso (INNOVATION / ALTERNATIVE).
- `created_at` (LocalDateTime): timestamp de creación.
- `start_date`, `end_date` (LocalDate): rango de fechas opcional.
- `franchises` (String): lista de franquicias concatenadas.
- `file_size` (Long): tamaño del fichero en bytes.
- `content` (byte[]): contenido binario del Excel (`bytea` en PostgreSQL).

Decisiones de diseño

- **Uso de `bytea` para `content`:** se eligió almacenar el contenido binario directamente en la base de datos en lugar de usar un sistema de ficheros externo para simplificar la gestión de backups, mantener atomicidad transaccional y facilitar la re-descarga sin dependencias externas.
- **Enumerados como `VARCHAR`:** los tipos `report_type` y `report_category` se almacenan como strings (`@Enumerated(EnumType.STRING)`) para legibilidad y evitar dependencia de ordinales que podrían cambiar.
- **Firma de duplicidad calculada:** no se almacena un campo de firma en la tabla. En su lugar, el `ProbeService` calcula dinámicamente la firma (hash MD5 de los parámetros) y busca informes recientes con los mismos valores de tipo, categoría, franquicias y fechas.
- **Índices:** se creó un índice compuesto en (`report_type`, `report_category`, `created_at`) para optimizar consultas de duplicidad y listado de historial.

Justificación del esquema y uso de Flyway

El uso de un esquema dedicado (`alsea_reports`) aísla los datos del Report Service del resto de esquemas de BIC, evitando conflictos de nombres y facilitando migraciones independientes. Flyway se eligió por:

- Versionado automático de cambios en el esquema mediante scripts SQL numerados.
- Aplicación idempotente de migraciones (solo se ejecutan scripts nuevos).
- Trazabilidad completa en tabla de metadatos de Flyway (`flyway_schema_history`).
- Integración nativa con Spring Boot mediante configuración en `application.properties`.

Cuadro 5.1: Atributos de la entidad *Report* en `alsea_reports.report`.

Campo (columna)	Tipo lógico (Java / SQL)	Nulo	Descripción
<code>id</code>	UUID / UUID	No	Identificador único del informe (clave primaria).
<code>file_name</code>	String / varchar(512)	No	Nombre del fichero Excel generado (incluye extensión).
<code>report_type</code>	enum ReportType / varchar	No	Tipo de informe: KPI o PRODUCT.
<code>report_category</code>	enum ReportCategory / varchar	No	Categoría de proceso: INNOVATION o ALTERNATIVE.
<code>created_at</code>	LocalDateTime / timestamp	No	Fecha y hora de creación del informe.
<code>start_date</code>	LocalDate / date	Sí	Fecha de inicio del rango temporal (opcional).
<code>end_date</code>	LocalDate / date	Sí	Fecha de fin del rango temporal (opcional).
<code>franchises</code>	String / text	No	Franquicia(s) seleccionadas; se almacena como cadena (p. ej., lista separada por comas).
<code>file_size</code>	Long / bigint	No	Tamaño del fichero Excel en bytes (útil para diagnóstico y control).
<code>content</code>	byte[] / bytea	No	Contenido binario del Excel (workbook).

5.3 Interacción cliente–servidor

Esta sección detalla los flujos de comunicación entre el cliente Angular y el backend Spring Boot, describiendo los principales casos de uso mediante diagramas de secuencia y especificaciones de endpoints.

5.3.1 Endpoints REST

La API expone los siguientes endpoints principales:

- **POST /api/reports/generate**

- **Descripción:** inicia la generación de un nuevo informe.

- **Request body** (JSON):

```
{
  "reportType": "KPI" | "PRODUCT",
  "reportCategory": "INNOVATION" | "ALTERNATIVE",
  "franchises": ["Starbucks", "Burger King", ...],
  "startDate": "2024-01-01", // opcional
  "endDate": "2024-12-31"    // opcional
}
```

- **Response:** stream binario del Excel (200 OK) o mensaje de error/duplicado (409 Conflict, 400 Bad Request).

- **POST /api/reports/probe**

- **Descripción:** verifica si existe un informe duplicado reciente.

- **Request body:** mismos parámetros que /generate.

- **Response** (JSON):

```
{
  "isDuplicate": true,
  "reportId": "a1b2c3d4-...",
  "createdAt": "2024-11-15T10:30:00"
}
```

- **GET /api/reports/history?page=0&size=5**

- **Descripción:** devuelve listado paginado de informes.

- **Response** (JSON):

```
{
  "content": [
    {
```

```
        "id": "uuid",
        "fileName": "KPI_Innovation_2024-11-15.xlsx",
        "reportType": "KPI",
        "reportCategory": "INNOVATION",
        "createdAt": "2024-11-15T10:30:00",
        "franchises": "Starbucks,Burger King",
        "fileSize": 1048576
    },
    ...
],
"totalElements": 42,
"totalPages": 9,
"number": 0
}
```

- **GET /api/reports/{id}/download**

- **Descripción:** descarga un informe por su ID.
- **Response:** stream binario del Excel (200 OK) o 404 Not Found.

5.3.2 Manejo de errores

El backend devuelve códigos [HTTP](#) estándar y mensajes estructurados:

- **200 OK:** operación exitosa.
- **201 Created:** informe generado correctamente.
- **400 Bad Request:** parámetros inválidos (fechas incorrectas, campos obligatorios ausentes).
- **404 Not Found:** informe solicitado no existe.
- **409 Conflict:** intento de generar un informe duplicado reciente.
- **500 Internal Server Error:** error no controlado en el servidor.

Los mensajes de error incluyen un campo `message` descriptivo y un código de error interno para facilitar el debugging:

```
{  
  "error": "DUPLICATE_REPORT",  
  "message": "Un informe idéntico fue generado hace 2 minutos.",  
  "reportId": "uuid"  
}
```

Implementación y pruebas

6.1 Tareas de mejora

Durante el desarrollo del proyecto, se abordaron diversas tareas de mejora que no solo añadieron nuevas funcionalidades, sino que también optimizaron la calidad, seguridad y mantenibilidad del código existente. Estas tareas se clasificaron en diferentes categorías según su naturaleza y prioridad, y se gestionaron a través del sistema de seguimiento Jira como parte del [EPIC](#) "Improvements for ALSEA Project in 2025 Q3 & Q4".

Las tareas de mejora se distribuyeron a lo largo de los tres meses de prácticas, abordando aspectos críticos como la seguridad de las dependencias, la cobertura de pruebas, la refactorización del código y la eliminación de duplicaciones. A continuación, se describen en detalle las principales categorías de tareas de mejora abordadas durante el proyecto.

6.1.1 Vulnerabilidades

El *frontend* presentaba inicialmente múltiples vulnerabilidades ([CVE](#)) asociadas a sus dependencias. En un primer momento, el origen del problema se debía en parte a la coexistencia de los gestores de paquetes *npm* y *Yarn*, lo que generaba inconsistencias en el árbol de dependencias y diferentes resoluciones de versiones según el gestor empleado. Una vez unificada la gestión en *Yarn*, las vulnerabilidades no desaparecieron por completo, ya que la versión de *Angular* utilizada (15) era considerablemente anterior, lo que limitaba el acceso a versiones parcheadas de algunas librerías, tal y como se recoge en la documentación técnica.

Estrategia adoptada:

1. **Estandarización en Yarn:** se eliminaron los ficheros `package-lock.json`, se configuró *Yarn* como gestor único en los archivos `angular.json` y `package.json`,

y se realizó un proceso de deduplicación con `yarn-deduplicate` seguido de una reinstalación completa de dependencias. Esta medida permitió estabilizar el entorno de desarrollo y homogeneizar la instalación en local y en [CI](#).

2. **Mitigación mediante `resolutions`**: dado que la actualización a *Angular 16+* quedaba fuera del alcance del proyecto, se optó por emplear el bloque `resolutions` de *Yarn* para forzar versiones parcheadas de dependencias transitivas, sobrescribiendo las versiones vulnerables solicitadas por los paquetes padre. Esta técnica permitió corregir gran parte de los avisos de seguridad sin alterar el núcleo del framework.

Vulnerabilidades mitigadas: `webpack-dev-middleware` (HIGH, Path Traversal), `path-to-regexp` (HIGH), `webpack`, `postcss`, `@babel/runtime` (MODERATE) y `tmp` (LOW).

Situación final: tras la estandarización y las `resolutions`, el número de vulnerabilidades se redujo significativamente, quedando solo tres *CVEs* menores vinculadas a dependencias del tooling de *Angular*. Dado que su impacto era limitado y no afectaba al entorno de ejecución, se decidió priorizar la integración funcional completa entre el *frontend* y el *backend* actualizado con las nuevas funcionalidades anteriormente comentadas. Como trabajo futuro se plantea la actualización del proyecto a *Angular 16+*, lo que permitiría eliminar el bloque de `resolutions` y resolver de forma nativa las vulnerabilidades remanentes.

6.1.2 Coverage

6.1.3 Refactorización

6.1.4 Duplicación

6.2 Spring Batch discussion dificultades técnicas

6.3 Pruebas

6.3.1 Testing unitario

6.3.2 Testing de integración

6.3.3 Pruebas manuales y entorno de [QA](#)

Para cada tarea asignada en Jira, se seguía un flujo de validación manual en un entorno de pruebas controlado antes de integrar los cambios en las ramas principales. El proceso de [QA](#) se apoyaba en la infraestructura de Ansible gestionada mediante [AWX](#) y el registry de

imágenes Quay.

Flujo de trabajo de QA:

1. **Desarrollo y commits:** tras completar la implementación de una funcionalidad o corrección, se realizaban los commits necesarios en la rama feature correspondiente siguiendo las convenciones de Gitflow.
2. **Actualización de versión:** se actualizaba el número de versión en los archivos de configuración (`package.json` en frontend, `pom.xml` en backend) para reflejar los cambios introducidos y facilitar la trazabilidad.
3. **Build y publicación de imagen:** el pipeline de GitHub Actions construía automáticamente una nueva imagen Docker del servicio modificado y la publicaba en Quay con la etiqueta de versión correspondiente.
4. **Despliegue mediante AWX:** desde la interfaz web de AWX, se ejecutaba el playbook de Ansible correspondiente que levantaba el entorno de pruebas (máquina virtual con todos los servicios necesarios: base de datos PostgreSQL, Elasticsearch, backend, frontend). El playbook descargaba la nueva imagen desde Quay y la desplegaba en el entorno aislado.
5. **Validación funcional:** una vez levantado el entorno, se accedía mediante Google Chrome a la aplicación desplegada y se realizaban pruebas manuales exhaustivas: generación de informes con diferentes parámetros, verificación del historial, comprobación del control de duplicidad, validación de mensajes de error, etc.
6. **Iteración o cierre:** si se detectaban problemas durante las pruebas, se volvía al paso 1 para corregirlos. Si todas las validaciones eran exitosas, se marcaba la tarea como QA OK en Jira y se procedía a crear el PR hacia development.

Este enfoque garantizaba que cada cambio se validara en un entorno realista antes de su integración, reduciendo significativamente la aparición de defectos en las ramas principales y asegurando la calidad del producto final.

Solución desarrollada

7.1 Introducción

En este capítulo se describe la solución desarrollada durante el proyecto, enfocada en la modernización y mejora de la interfaz de usuario del Report Service. A diferencia del sistema original, que ofrecía únicamente una pantalla básica de generación de informes, la nueva solución incorpora funcionalidades avanzadas que mejoran significativamente la experiencia del usuario y las capacidades del sistema.

Las principales mejoras implementadas incluyen:

- Una interfaz renovada para el generador de informes con retroalimentación en tiempo real sobre el estado del proceso.
- Un módulo completamente nuevo de historial que permite consultar y re-descargar informes previamente generados.
- Validaciones mejoradas y mensajes informativos que guían al usuario durante todo el flujo de trabajo.
- Control de duplicidad integrado que evita la regeneración innecesaria de informes idénticos.

A continuación, se detallan cada una de las páginas que componen la solución desarrollada.

7.2 Página de Generar

La página de generación de informes constituye el núcleo de la interfaz de usuario. Esta pantalla permite al usuario configurar todos los parámetros necesarios para crear un informe

personalizado y recibir retroalimentación inmediata sobre el proceso.

En una primera fase de diseño, se había planteado la existencia de tres interfaces principales: una pantalla inicial a modo de menú de inicio desde la que el usuario podría acceder tanto al historial de informes como a la página de generación. Sin embargo, tras una discusión dentro del equipo, se concluyó que dicha estructura añadía una capa innecesaria de navegación y no aportaba valor real al flujo de uso. Se decidió, por tanto, simplificar la experiencia del usuario manteniendo únicamente dos interfaces —*Generar informe* e *Historial*—, siendo la primera la principal y punto de entrada de la aplicación.

7.2.1 Funcionalidad

- Permite configurar los parámetros del informe.
- Muestra notificaciones en tiempo real del estado de la generación.
- Descarga automática del informe al finalizar.
- Incluye un botón para navegar al historial.

7.2.2 Diseño de la interfaz

La interfaz del generador se ha diseñado para minimizar la incertidumbre del usuario: los controles relevantes se deshabilitan mientras una generación está en curso, se muestran mensajes claros sobre el estado del proceso y se ofrece ayuda contextual para elegir parámetros. Se incorporan también medidas de accesibilidad y opciones de reintento o cancelación en caso de fallos transitorios. Cuando el sistema detecta un informe idéntico reciente, la UI sugiere al usuario re-descargar desde Historial en lugar de volver a generar, respetando la política de control de duplicidad del backend.

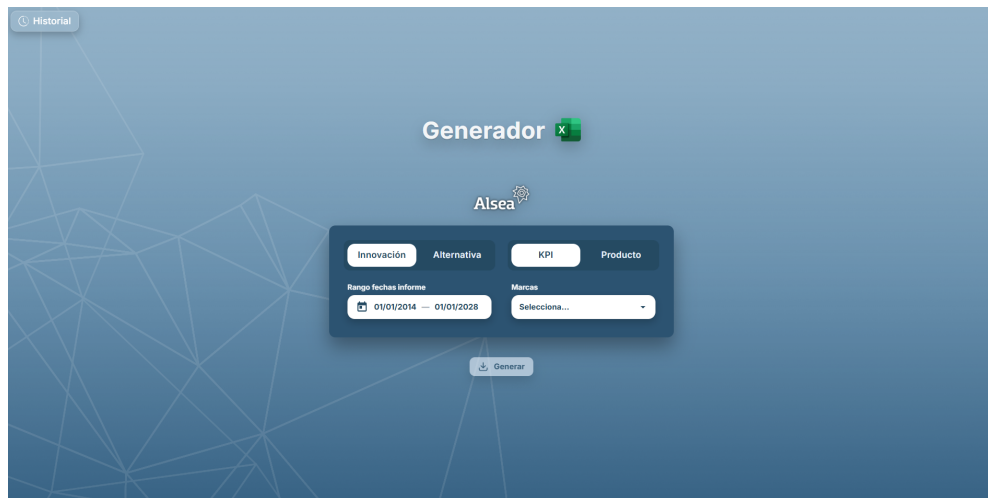


Figura 7.1: Interfaz principal del módulo de generación de informes.

7.2.3 Flujo de interacción

El flujo de trabajo implementado en la página de generación garantiza una experiencia coherente y eficiente. El siguiente diagrama ilustra el proceso completo desde que el usuario accede a la página hasta que puede re-descargar el informe desde el historial.

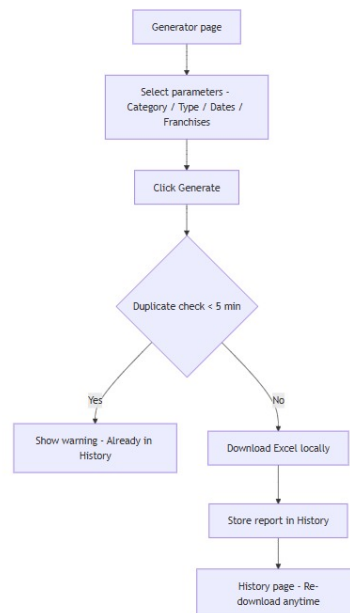


Figura 7.2: Diagrama de flujo del proceso de generación de informes.

Como se observa en la figura 7.2, el proceso comienza cuando el usuario selecciona los

parámetros del informe y pulsa el botón *Generar*. En ese momento, el sistema realiza una comprobación de duplicidad para verificar si existe un informe idéntico generado en los últimos 5 minutos. Si se detecta un duplicado, el sistema muestra un aviso informando al usuario de que el informe ya está disponible en el historial, evitando así una regeneración innecesaria. En caso contrario, el proceso continúa con la descarga local del Excel generado y el almacenamiento del informe en la base de datos. Finalmente, el usuario puede acceder en cualquier momento a la página de historial para re-descargar el informe sin necesidad de volver a generarlo.

7.3 Página de Historial

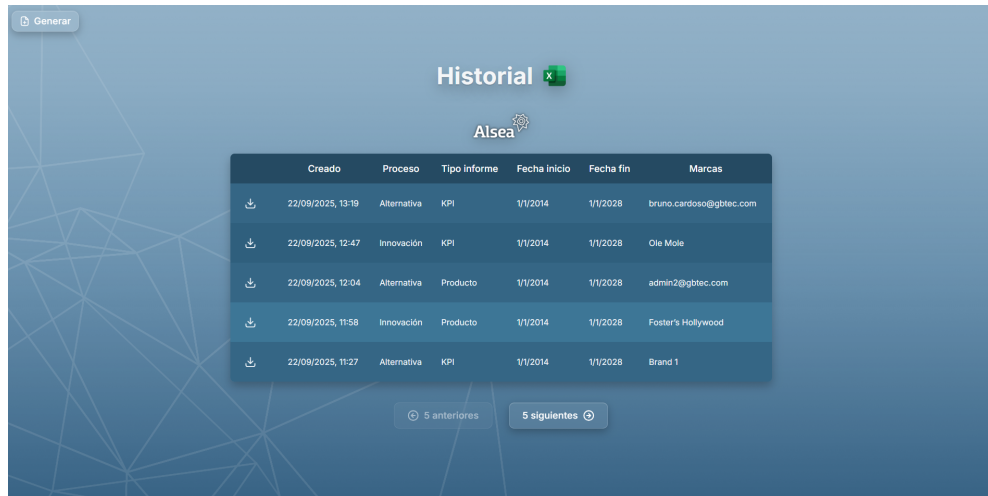
La página de historial es una funcionalidad completamente nueva desarrollada en este proyecto. Antes de esta mejora, el sistema no contaba con ningún mecanismo para consultar informes previamente generados, obligando a los usuarios a regenerarlos cada vez que necesitaban acceder a información pasada.

7.3.1 Funcionalidad

- Presenta una tabla con todos los informes generados.
- Cada fila incluye columnas de descarga, fecha de creación, tipo, categoría, franquicia(s), fechas y tamaño.
- Implementa paginación (*5 anteriores / 5 siguientes*).
- Los informes se ordenan de los más recientes a los más antiguos.
- Permite la re-descarga directa de cualquier informe almacenado.

7.3.2 Diseño de la interfaz

El diseño de la tabla de historial prioriza la claridad y la facilidad de uso. Cada informe muestra toda la información relevante de forma estructurada, permitiendo al usuario identificar rápidamente el documento que necesita. El botón de descarga en cada fila facilita el acceso inmediato al contenido sin necesidad de regenerar el informe.



Creado	Proceso	Tipo informe	Fecha inicio	Fecha fin	Marcas
22/09/2025, 13:19	Alternativa	KPI	1/1/2014	1/1/2028	bruno.cardoso@gbtec.com
22/09/2025, 12:47	Innovación	KPI	1/1/2014	1/1/2028	Ole Mole
22/09/2025, 12:04	Alternativa	Producto	1/1/2014	1/1/2028	admin2@gbtec.com
22/09/2025, 11:58	Innovación	Producto	1/1/2014	1/1/2028	Foster's Hollywood
22/09/2025, 11:27	Alternativa	KPI	1/1/2014	1/1/2028	Brand 1

Figura 7.3: Vista de la página de historial de informes generados.

7.4 Release y publicación

Una vez completado el desarrollo de las funcionalidades descritas, con las interfaces validadas y aprobadas por el equipo, y habiendo superado todas las pruebas de QA en el entorno de pruebas gestionado mediante AWWX, el último paso consistía en preparar y publicar la nueva versión del Report Service siguiendo el proceso formal de release establecido por la empresa.

Este proceso seguía las convenciones de Gitflow y las políticas de versionado de GBTEC, garantizando la trazabilidad completa y minimizando riesgos de despliegue:

1. **Creación de la rama release candidate (RC):** desde la rama `development`, que contenía todos los cambios validados e integrados, se creaba una nueva rama `release/vX.Y.Z-rc` (por ejemplo, `release/v2.1.0-rc`). Esta rama servía como candidata para la release y permitía realizar ajustes finales sin bloquear el desarrollo de nuevas funcionalidades en `development`.
2. **Version bump:** se actualizaban los números de versión en los archivos de configuración de ambos proyectos:
 - Backend: actualización del elemento `<version>` en el archivo `pom.xml` de Maven.
 - Frontend: actualización del campo `version` en el archivo `package.json` gestionado por Yarn.

Estos cambios se comprometían en la rama RC con un mensaje descriptivo (e.g., `"chore: bump version to 2.1.0"`).

3. **Builds y validaciones finales:** se ejecutaban los pipelines de GitHub Actions para compilar el proyecto completo, ejecutar todas las pruebas (unitarias e integradas) y verificar las métricas de calidad en SonarCloud. Se realizaba una última ronda de validaciones manuales en el entorno de QA para confirmar que no existían regresiones.
4. **Creación del tag Git:** una vez aprobadas todas las validaciones, se creaba un tag Git con el número de versión (e.g., v2 . 1 . 0) apuntando al último commit de la rama RC. Este tag marcaba formalmente la versión publicable y permitía rastrear exactamente qué código se desplegó en producción.
5. **Merge a main:** se creaba un PR desde la rama RC hacia la rama main, que representaba el estado estable de producción. Tras la revisión final y aprobación por parte del equipo técnico, se integraban los cambios mediante merge commit preservando el historial completo.
6. **Merge a development:** paralelamente, se integraban los cambios de la rama RC de vuelta a development para sincronizar cualquier ajuste realizado durante el proceso de release (como el version bump o correcciones de última hora), garantizando que development reflejara siempre el estado más actualizado.
7. **Despliegue en producción:** el equipo de DevOps utilizaba el tag de versión para desplegar los artefactos correspondientes (imágenes Docker desde Quay, paquetes JAR desde JFrog) en el entorno de producción de ALSEA.
8. **Documentación:** se actualizaban los archivos README con notas de la release, se documentaban los cambios principales (changelog) y se comunicaba al cliente la disponibilidad de la nueva versión con sus mejoras y funcionalidades.

Con este proceso, la nueva versión del Report Service quedó lista para su uso en producción, incorporando todas las mejoras de almacenamiento histórico, procesamiento asíncrono, control de duplicidad y experiencia de usuario modernizada.

Conclusión y trabajo futuro

8.1 Conclusiones

Este Trabajo de Fin de Grado documenta la modernización del módulo Report Service en la plataforma BIC para ALSEA, desarrollado durante tres meses de prácticas en GBTEC Software AG. Los objetivos se cumplieron satisfactoriamente, transformando una herramienta funcional pero limitada en un sistema moderno y eficiente.

Entre los logros más significativos del proyecto destacan:

- **Implementación de almacenamiento histórico:** se desarrolló un sistema completo de persistencia de informes en PostgreSQL, utilizando Flyway para el versionado del esquema de base de datos. Esto ha eliminado la necesidad de regenerar informes cada vez que se requiere consultar información pasada, mejorando significativamente la eficiencia operativa.
- **Procesamiento asíncrono con Spring Batch:** la introducción de generación asíncrona mediante Spring Batch ha resuelto el problema de bloqueo de la interfaz, permitiendo que los usuarios continúen interactuando con la aplicación mientras se generan los informes. Esto ha mejorado notablemente la experiencia de usuario, especialmente en informes de gran tamaño.
- **Control de duplicidad:** el sistema de verificación de duplicados basado en firmas únicas ha demostrado ser efectivo para evitar la generación redundante de informes idénticos en ventanas temporales cortas, reduciendo la carga del servidor y optimizando el uso de recursos.
- **Interfaz de usuario modernizada:** el desarrollo de una nueva interfaz con retroalimentación en tiempo real y una página de historial completamente nueva ha transfor-

mado la experiencia de usuario, pasando de una interfaz anticuada y sin funcionalidades avanzadas a una herramienta moderna e intuitiva.

- **Mejora de la calidad del código:** las tareas de refactorización, reducción de duplicidad y aumento de cobertura de pruebas han mejorado sustancialmente la mantenibilidad del código, facilitando futuras evoluciones del sistema.
- **Gestión de vulnerabilidades:** se logró reducir significativamente el número de vulnerabilidades de seguridad en el frontend mediante la estandarización en Yarn y el uso estratégico de `resolutions`, equilibrando seguridad con estabilidad del sistema.

Desde el punto de vista del aprendizaje personal, este proyecto ha proporcionado una experiencia invaluable en el desarrollo de software empresarial real, permitiendo aplicar conocimientos teóricos de la titulación en un contexto profesional. La metodología ágil empleada, las revisiones de código, la gestión de tareas en Jira y el trabajo con tecnologías modernas como Spring Batch, Angular, PostgreSQL y Elasticsearch han enriquecido significativamente la formación práctica.

Además, el proyecto ha puesto de manifiesto la importancia de equilibrar diferentes aspectos del desarrollo de software: funcionalidad, seguridad, rendimiento, mantenibilidad y experiencia de usuario. Las decisiones tomadas durante el proyecto, como priorizar la funcionalidad sobre la resolución de vulnerabilidades menores, ilustran el tipo de compromisos que se deben gestionar en entornos de desarrollo reales con limitaciones de tiempo y recursos.

En conclusión, el Report Service modernizado representa una mejora sustancial sobre el sistema original, proporcionando a ALSEA una herramienta más potente, fiable y escalable para la generación y gestión de informes ejecutivos. El proyecto ha cumplido sus objetivos técnicos y formativos, dejando una base sólida para futuras ampliaciones y mejoras del sistema.

8.2 Posibles ampliaciones futuras

Aunque el proyecto ha logrado cumplir con los objetivos planteados, existen diversas líneas de trabajo que podrían abordarse en el futuro para seguir mejorando el Report Service. Estas ampliaciones se han identificado durante el desarrollo del proyecto y representan oportunidades para aumentar la funcionalidad, seguridad y usabilidad del sistema.

8.2.1 Actualización de la versión mayor de Angular

Como se mencionó en el apartado de vulnerabilidades, una de las principales mejoras pendientes es la actualización de Angular de la versión 15 a una versión más reciente. Esta actualización permitiría:

- Eliminar el bloque de `resolutions` del `package.json`, permitiendo que las dependencias se actualicen automáticamente a versiones seguras.
- Resolver las tres vulnerabilidades restantes que no pudieron mitigarse sin cambiar de versión mayor.
- Aprovechar las nuevas funcionalidades y mejoras de rendimiento de versiones más recientes de Angular.
- Reducir los avisos de incompatibilidad que actualmente genera el uso de `resolutions`.

Sin embargo, esta actualización requiere un proceso de migración cuidadoso, siguiendo las guías oficiales de Angular para cada versión mayor, lo que implica una dedicación considerable de tiempo y pruebas exhaustivas.

8.2.2 Mejoras en la gestión del historial

Una de las funcionalidades más solicitadas para futuras versiones es la ampliación de las capacidades de gestión del historial de informes. Específicamente, se ha identificado la necesidad de implementar:

Eliminación de informes

Actualmente, todos los informes generados se almacenan indefinidamente en la base de datos. Sería deseable permitir a los usuarios eliminar informes que ya no necesiten, liberando espacio de almacenamiento y facilitando la gestión del historial.

El diseño preliminar de esta funcionalidad incluiría:



		Creado	Proceso	Tipo informe	Fecha inicio	Fecha fin	Marcas
		30/09/2025, 14:32	Alternativa	KPI	1/1/2014	1/1/2028	admin2@gbtec.com, admin@gbtec.com, Brand 1, bruno.cardoso@gbtec.com
		30/09/2025, 14:32	Innovación	KPI	1/1/2014	1/1/2028	Domino's Pizza
		30/09/2025, 10:55	Alternativa	Producto	1/1/2025	1/6/2025	Brand 1
		30/09/2025, 10:54	Alternativa	KPI	1/1/2025	1/6/2025	admin2@gbtec.com
		30/09/2025, 10:54	Innovación	KPI	1/1/2025	1/6/2025	Domino's Pizza

Figura 8.1: Mockup de la interfaz de historial con funcionalidad de eliminación de informes.

Como se observa en la figura 8.1, se añadiría una columna adicional con botones de eliminación para cada informe. Al pulsar el botón de borrar, aparecería un diálogo de confirmación para evitar eliminaciones accidentales.

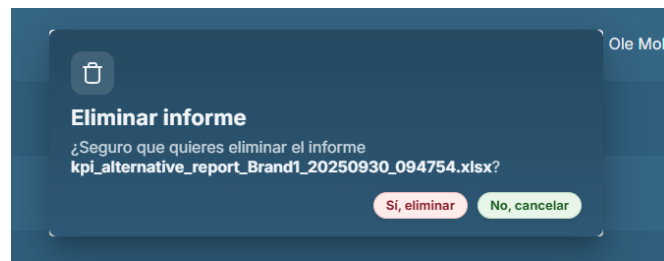


Figura 8.2: Diálogo de confirmación para la eliminación de informes.

La figura 8.2 muestra el diseño del diálogo de confirmación que solicitaría al usuario verificar la acción antes de eliminar permanentemente un informe del sistema.

Otras mejoras planificadas para el historial

Además de la funcionalidad de eliminación, se han identificado otras mejoras deseables:

- **Selección múltiple:** implementar checkboxes para seleccionar varios informes y realizar acciones en lote (eliminar múltiples informes, descargar varios informes simultáneamente).
- **Filtros avanzados:** añadir filtros por categoría, tipo, franquicia, rango de fechas, etc., para facilitar la búsqueda de informes específicos en historiales extensos.
- **Ordenación personalizable:** permitir ordenar la tabla por diferentes columnas (fecha, tamaño, tipo, categoría).

- **Exportación del listado:** posibilidad de exportar el listado del historial a CSV o Excel para auditorías o análisis externos.

8.2.3 Refactorización adicional del código backend

Aunque se realizaron importantes tareas de refactorización durante el proyecto que hicieron que se mejorara las métricas de calidad del código, siempre existe margen para mejorar. Las áreas identificadas para futuras mejoras incluyen:

- Mayor modularización de utilidades comunes.
- Ampliación de la cobertura de pruebas unitarias e integradas.
- Optimización de consultas a la base de datos.
- Implementación de patrones de diseño adicionales para mejorar la mantenibilidad.

8.2.4 Funcionalidades avanzadas

Mirando más allá de las mejoras inmediatas, se podrían considerar funcionalidades más ambiciosas:

- **Programación de informes:** permitir a los usuarios programar la generación automática de informes periódicos.
- **Notificaciones:** enviar notificaciones cuando un informe esté listo.
- **Plantillas personalizables:** permitir a los usuarios definir plantillas de informe con métricas específicas.
- **Dashboard analítico:** crear visualizaciones interactivas de las métricas de los informes sin necesidad de descargar Excel.
- **Control de acceso granular:** implementar permisos específicos por usuario o rol para la generación y consulta de informes.

Estas ampliaciones futuras representan oportunidades para seguir evolucionando el Report Service hacia una herramienta cada vez más completa y alineada con las necesidades cambiantes de ALSEA.

Apéndice

Material adicional

EXEMPLO de capítulo con formato de apéndice, onde se pode incluír material adicional que non teña cabida no corpo principal do documento, suxeito á limitación de 80 páxinas establecida no regulamento de TFGs.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque.

Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lista de acrónimos

API Application Programming Interface. 18, 21, 22, 27

AWX Interfaz web para la automatización con Ansible. 8, 20, 31, 32, 37

BPM Business Process Management. 2

CI Continuous Integration. 31

CVE Common Vulnerabilities and Exposures. 30

EAM Enterprise Architecture Management. 2

EPIC Conjunto de historias de usuario agrupadas en Jira. 8–10, 30

GRC Governance, Risk and Compliance. 2

HTTP HyperText Transfer Protocol. 16, 18, 19, 28

JPA Java Persistence API. 6, 19

JSON JavaScript Object Notation. 6, 18, 19, 23

KPI Key Performance Indicator. 2, 24

PR Pull Request. 8, 10, 32, 38

QA Quality Assurance. iv, 8, 31, 32, 37, 38

REST Representational State Transfer. 16, 18, 19, 21, 22, 27

SQL Structured Query Language. 6, 26

Udemy Plataforma de formación en línea. [10](#)

UUID Universally Unique Identifier. [19](#)

Glosario

Angular Framework de desarrollo web basado en TypeScript para la construcción de aplicaciones cliente (*frontend*) modulares y escalables.. 7

Ansible Herramienta de automatización de configuración y despliegue de aplicaciones que utiliza archivos YAML para definir tareas y flujos de trabajo.. 8

Aspose Cells Biblioteca de Java empleada para la generación y manipulación avanzada de archivos en formato Excel.. 6

Docker Plataforma de contenedores que permite empaquetar aplicaciones y sus dependencias en un entorno aislado y reproducible.. 10

Elasticsearch Motor de búsqueda y análisis distribuido basado en documentos JSON. En la BIC Platform forma parte del ciclo de vida de un proceso, ya que almacena e indexa las instancias y variables generadas durante su ejecución en Process Execution, permitiendo consultar y analizar el estado, métricas y resultados de los procesos en tiempo real.. 2

Flyway Herramienta de migración de bases de datos que garantiza el versionado y la coherencia de los esquemas en los diferentes entornos.. 9

Gitflow Convención y conjunto de prácticas para el uso de ramas en Git que organiza el desarrollo mediante ramas
textttfeature,
textttrelease,
texttt Hotfix y
textttdevelop/
textttmain, facilitando la gestión de releases y mantenimiento.. 8

GitHub Actions Plataforma de integración continua y entrega continua (CI/CD) integrada en GitHub que permite definir flujos de trabajo (workflows) para compilar, probar y desplegar aplicaciones mediante acciones reutilizables.. 8

Heatmap Representación gráfica de datos en la que los valores se representan mediante colores, facilitando la identificación de patrones y tendencias.. 2

Histograma Representación gráfica de la distribución de datos mediante barras verticales u horizontales, donde cada barra representa la frecuencia o cantidad de elementos en un rango o categoría determinada.. vi, 3

JFrog Artifactory Gestor universal de artefactos que almacena y gestiona paquetes, dependencias y artefactos binarios (imágenes Docker, paquetes Maven/NPM, etc.), facilitando el versionado y distribución en pipelines de CI/CD.. 8

Jira Herramienta de gestión ágil de proyectos que permite organizar tareas, historias de usuario y seguimiento de incidencias.. 8, 9

Kanban Metodología ágil de gestión visual del trabajo basada en tableros y tarjetas, que permite controlar el flujo de tareas en tiempo real.. 8

On-Premise Modelo de implementación de software en el que las aplicaciones y datos se alojan en los servidores locales de una organización, en lugar de utilizar servicios en la nube.. 20

PostgreSQL Sistema de gestión de bases de datos relacional y de código abierto empleado para almacenar información estructurada.. 7

Quay Registro de imágenes de contenedores (container registry) que permite almacenar, versionar y distribuir imágenes Docker en entornos empresariales.. 8

Retroplanning Técnica de planificación de proyectos que consiste en definir las tareas y hitos necesarios para alcanzar un objetivo, comenzando desde la fecha de finalización prevista y retrocediendo hasta el inicio del proyecto.. 2

SonarCloud Plataforma de análisis estático de código que ayuda a identificar problemas de calidad, vulnerabilidades y malas prácticas en el desarrollo de software.. 8

Spring Batch Framework de Spring orientado al procesamiento por lotes (*batch processing*), utilizado para ejecutar tareas asíncronas de larga duración.. 4

Bibliografía

- [1] V. Driessen, “A successful git branching model,” *nvie.com*, 2010, accedido: 15-11-2024. [En línea]. Disponible en: <https://nvie.com/posts/a-successful-git-branching-model/>
- [2] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland, and D. Thomas, “Manifesto for agile software development,” 2001, accedido: 15-11-2024. [En línea]. Disponible en: <https://agilemanifesto.org/>
- [3] D. J. Anderson, *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010.
- [4] K. Schwaber and J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. Scrum.org, 2020. [En línea]. Disponible en: <https://scrumguides.org/>